Exhaustiveness Checking

Concepts of Programming Languages

CAS CS 320

Recap

Pattern Matching (High Level)

```
let rec sum l =
  match l with
  | [] -> 0
  | x :: xs -> x + sum xs
```

Values come in different "shapes"

We want to do things based on the different shapes

Matching allows us to condition on the shape of the value

Patterns (High Level)

```
match x with
| 1 -> e1
| 2 -> e2
| _ -> e3
```

A pattern describes the "shape" of a **value**

But we don't *just* care about the "shape" we also want to do stuff with the pieces of the data

<u>Patterns</u> are syntactic representations of values with names

Shallow vs. Deep Pattern Matching

```
let third l =
  match l with
  | _ :: _ :: x :: _ -> Some x
  | _ -> None
```

We're interested implementing *deep* pattern matching, which allows for *nested* patterns

We can also implement *shallow* pattern matching ADTs (which are sometimes called *eliminators*)

Match Syntax

```
<expr> ::= match <expr> with <arms>
<arms> ::= <arm> <arms> | <arm>
 <arm> ::= | <pat> -> <expr>
 <pat> ::= () | true | false
         | [] | <pat> :: <pat>
         | <pat> , <pats>
         | <int> | <var> | ( <pat> )
<pats> ::= <pat> , <pats> | <pat>
```

Pattern Typing Judgments

$$p : \tau \Rightarrow \Gamma$$

"The pattern p is of type τ and generates the variable declarations Γ"

Our derivations now have multiple kinds of judgments

Match Typing

$$\frac{}{\texttt{()} : \texttt{unit} \Rightarrow \varnothing}\ \textsc{UnitP} \qquad \frac{}{\texttt{true} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{TrueP} \qquad \frac{}{\texttt{false} : \texttt{bool} \Rightarrow \varnothing}\ \textsc{FalseP}$$

$$\frac{\text{n is an integer literal pattern}}{\mathsf{n} : \texttt{int} \Rightarrow \varnothing}\ \textsc{IntLitP} \qquad \frac{x \text{ is a variable pattern}}{x : \tau \Rightarrow \{x : \tau\}}\ \textsc{VarP} \qquad \frac{}{\texttt{[]} : \texttt{int list} \Rightarrow \varnothing}\ \textsc{NilP}$$

$$\frac{p_1 : \texttt{int} \Rightarrow \Gamma_1 \qquad p_2 : \texttt{int list} \Rightarrow \Gamma_2 \qquad \Gamma_1 \text{ and } \Gamma_2 \text{ are disjoint}}{p_1 \texttt{ :: } p_2 : \texttt{int list} \Rightarrow \Gamma_1, \Gamma_2}\ \textsc{ConsP}$$

$$\frac{p_1 : \tau_1 \Rightarrow \Gamma_1 \qquad \ldots \qquad p_k : \tau_k \Rightarrow \Gamma_k \qquad \Gamma_1, \ldots, \Gamma_k \text{ are disjoint}}{p_1 \texttt{ , } \ldots \texttt{ , } p_k : \tau_1 \texttt{ * } \ldots \texttt{ * } \tau_k \Rightarrow \Gamma_1, \ldots, \Gamma_k}\ \textsc{TupleP}$$

$$\frac{\Gamma \vdash e_0 : \tau_0 \qquad p_1 : \tau_0 \Rightarrow \Gamma_1 \qquad \Gamma, \Gamma_1 \vdash e_1 : \tau \qquad \ldots \qquad p_k : \tau_0 \Rightarrow \Gamma_k \qquad \Gamma, \Gamma_k \vdash e_k : \tau}{\Gamma \vdash \texttt{match } e \texttt{ with } \texttt{|}\ p_1 \texttt{ -> } e_1 \ldots \texttt{ |}\ p_k \texttt{ -> } e_k : \tau}\ \textsc{Match}$$

Pattern matching judgment

$$v \text{ matches } p \Rightarrow \mathscr{E}$$

"The value v matches the pattern p and generates the bindings $\mathscr{E}$"

Again, our derivations now have multiple kinds of judgments

Match Semantics

$$\frac{}{\texttt{()} \text{ matches } \texttt{()} \Rightarrow \varnothing}\ \textsc{unitM} \qquad \frac{}{\top \text{ matches } \texttt{true} \Rightarrow \varnothing}\ \textsc{trueM} \qquad \frac{}{\bot \text{ matches } \texttt{false} \Rightarrow \varnothing}\ \textsc{falseM}$$

$$\frac{\text{n is an integer pattern for } n}{n \text{ matches n} \Rightarrow \varnothing}\ \textsc{intM} \qquad \frac{x \text{ is an variable pattern}}{v \text{ matches } x \Rightarrow \{x \mapsto v\}}\ \textsc{varM} \qquad \frac{}{[] \text{ matches } \texttt{[]} \Rightarrow \varnothing}\ \textsc{nilM}$$

$$\frac{n \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad [n_1, \ldots, n_k] \text{ matches } p_2 \Rightarrow \mathcal{E}_2}{[n, n_1, \ldots, n_k] \text{ matches } p_1 \texttt{ :: } p_2 \Rightarrow \mathcal{E}_1 \cup \mathcal{E}_2}\ \textsc{consM}$$

$$\frac{\mathcal{E} \vdash v_1 \text{ matches } p_1 \Rightarrow \mathcal{E}_1 \qquad \ldots \qquad \mathcal{E} \vdash v_k \text{ matches } p_k \Rightarrow \mathcal{E}_k}{\mathcal{E} \vdash v_1 \texttt{ , } \ldots \texttt{ , } v_k \text{ matches } p_1 \texttt{ , } \ldots \texttt{ , } p_k \Rightarrow \mathcal{E}_1 \cup \cdots \cup \mathcal{E}_k}\ \textsc{tupleM}$$

$$\frac{\mathcal{E} \vdash e_0 \Downarrow v_0 \qquad \forall j < i (v_0 \text{ does not match } p_j) \qquad v_0 \text{ matches } p_i \Rightarrow \mathcal{E}' \qquad \mathcal{E}'' = \text{merge}(\mathcal{E}, \mathcal{E}') \qquad \mathcal{E}'' \vdash e_i \Downarrow v}{\mathcal{E} \vdash \texttt{match } e_0 \texttt{ with } \texttt{|} \ p_1 \texttt{ -> } e_1 \ldots \texttt{ |} \ p_k \texttt{ -> } e_k \Downarrow v}\ \textsc{matchE}$$

Summary

We can formally define the syntax, typing and semantics of **match expressions**

Deep pattern matching requires the introduction of **patterns**

Practice Problem

```
match 1 :: 2 :: [] with
| [] -> 0
| _ :: x :: [] -> x
```

Give typing and semantic derivations for the above expression (do you're best with space)

Exhaustiveness Checking

The Problem

```
match 1 :: 2 :: [] with
| [] -> 0
| _ :: x :: [] -> x
```

We'd like to know not just that our matches are well-typed, but also that we *cover all our bases*

Warnings for Pattern Matching

There are two kinds of warnings we can get:

» **Non-exhaustive.** There is a possible input that will not be matched

» **Useless.** There is a pattern that will never be used

We can determine both of these by a kind of *reachability analysis* which makes heavy use of *decision trees*

Useful Patterns

Exhaustiveness

Uselessness

Restricted Setting

Useful Patterns (Recursive Definition)

Examples